DROP LOTTERY

Децентрализованная лотерея на Ethereum с использованием смарт-контракта и веб-интерфейса.

Агафонов Никита БПИ234

Суть приложения

Приложение позволяет пользователю подключить кошелёк, войти в розыгрыш и участвовать в децентрализованной лотерее.

Контракт принимает ставки, хранит список участников и выбирает победителя случайным образом.

Интерфейс показывает статус лотереи, таймер, призовой фонд, количество участников и журнал событий.

Почему был выбран этот стек

Для проекта был выбран стек, который покрывает сразу две части приложения: блокчейн-логику и пользовательский интерфейс.

Solidity и Hardhat используются для смарт-контрактов, React и TypeScript для фронтенда, Chainlink для честной случайности и автоматизации, а Ethers.js для связи интерфейса с блокчейном.

Solidity

Язык Solidity использовался для написания смарт-контракта лотереи.

Именно в контракте реализованы вход в розыгрыш, проверка условий, выбор победителя и перевод выигрыша. По сравнению с Vyper Solidity используется чаще, имеет больше документации.

Hardhat

Hardhat использовался для компиляции, тестирования, запуска локальной сети и деплоя контракта.

Это основной инструмент разработки для локальной отладки смарт-контрактов.

По сравнению с Truffle Hardhat современнее и удобнее, а по сравнению с Foundry лучше вписывается в TypeScript-окружение проекта.

Chainlink VRF

Chainlink VRF использовался для получения случайного числа при выборе победителя. Это нужно для честного розыгрыша без возможности манипулировать результатом. По сравнению с псевдослучайностью через block.timestamp или blockhash этот подход надёжнее и безопаснее.

Chainlink Automation

Chainlink Automation нужен для автоматического запуска розыгрыша, когда прошло нужное время и есть участники.

Это избавляет от ручного запуска или отдельный сервер.

По сравнению с обычным cron-скриптом это решение ближе к децентрализованной логике приложения.

Ethers.js

Ethers.js использовался для взаимодействия фронтенда и скриптов с контрактом.

С его помощью приложение читает данные, отправляет транзакции и слушает события контракта.

По сравнению с web3.js библиотека легче, современнее и удобнее для TypeScript.

Vite

Vite использовался для запуска и сборки фронтенда.

Дает быстрый старт dev-сервера и простую конфигурацию.

По сравнению с Webpack его проще настроить для небольшого React проекта.

Tailwind CSS

Tailwind CSS использовался для оформления интерфейса.

Он помог сделать ретро футуризм)

По сравнению с Bootstrap он дает больше свободы в дизайне и меньше шаблонности.

Zustand

Zustand использовался для хранения состояния лотереи и событий на клиенте.

Он помог удобно управлять данными без перегруженной архитектуры.

По сравнению с Redux Zustand проще и требует меньше шаблонного кода.

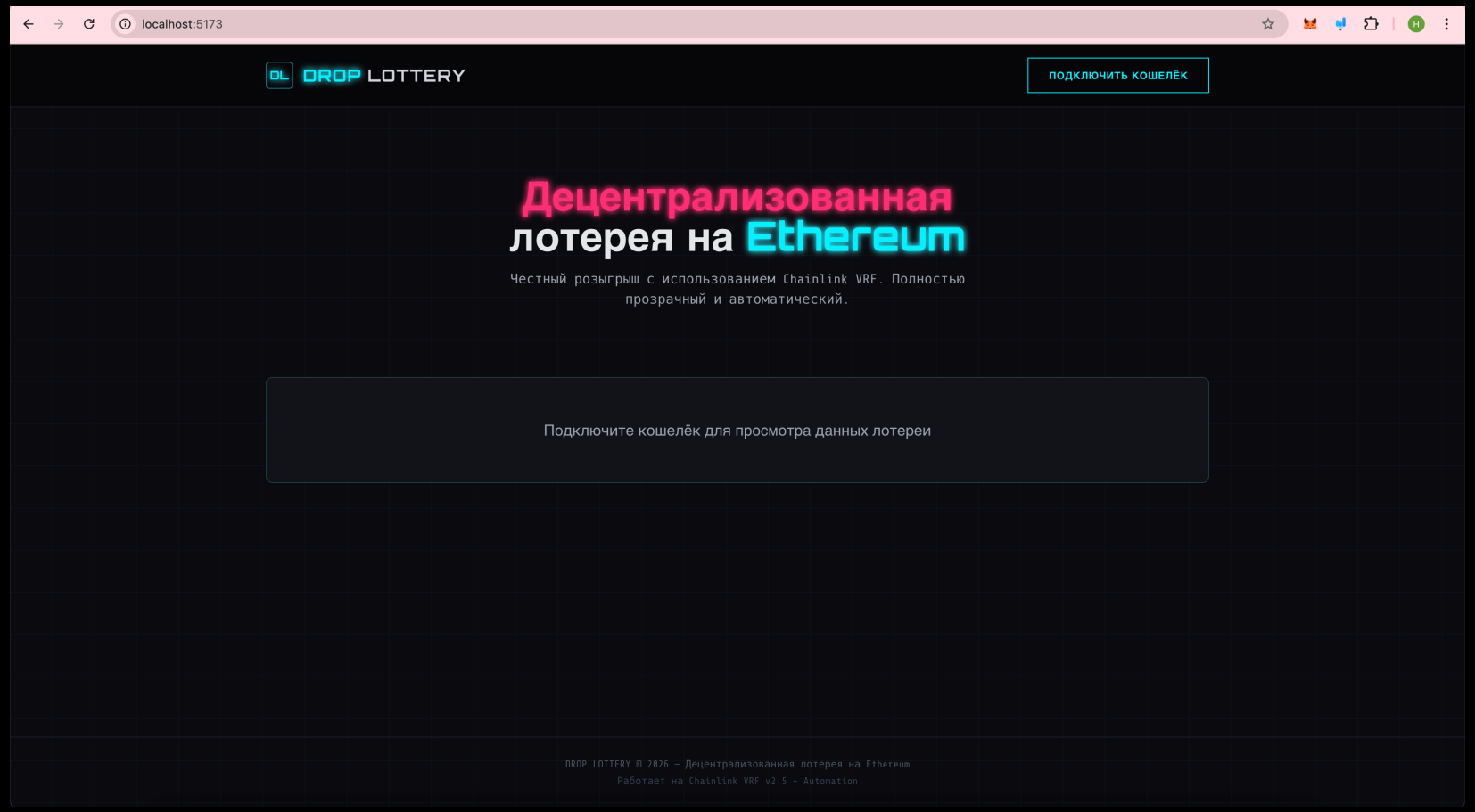
React Router и React Hot Toast

React Router использовался для маршрутизации страниц.

React Hot Toast применялся для уведомлений о транзакциях и ошибках.

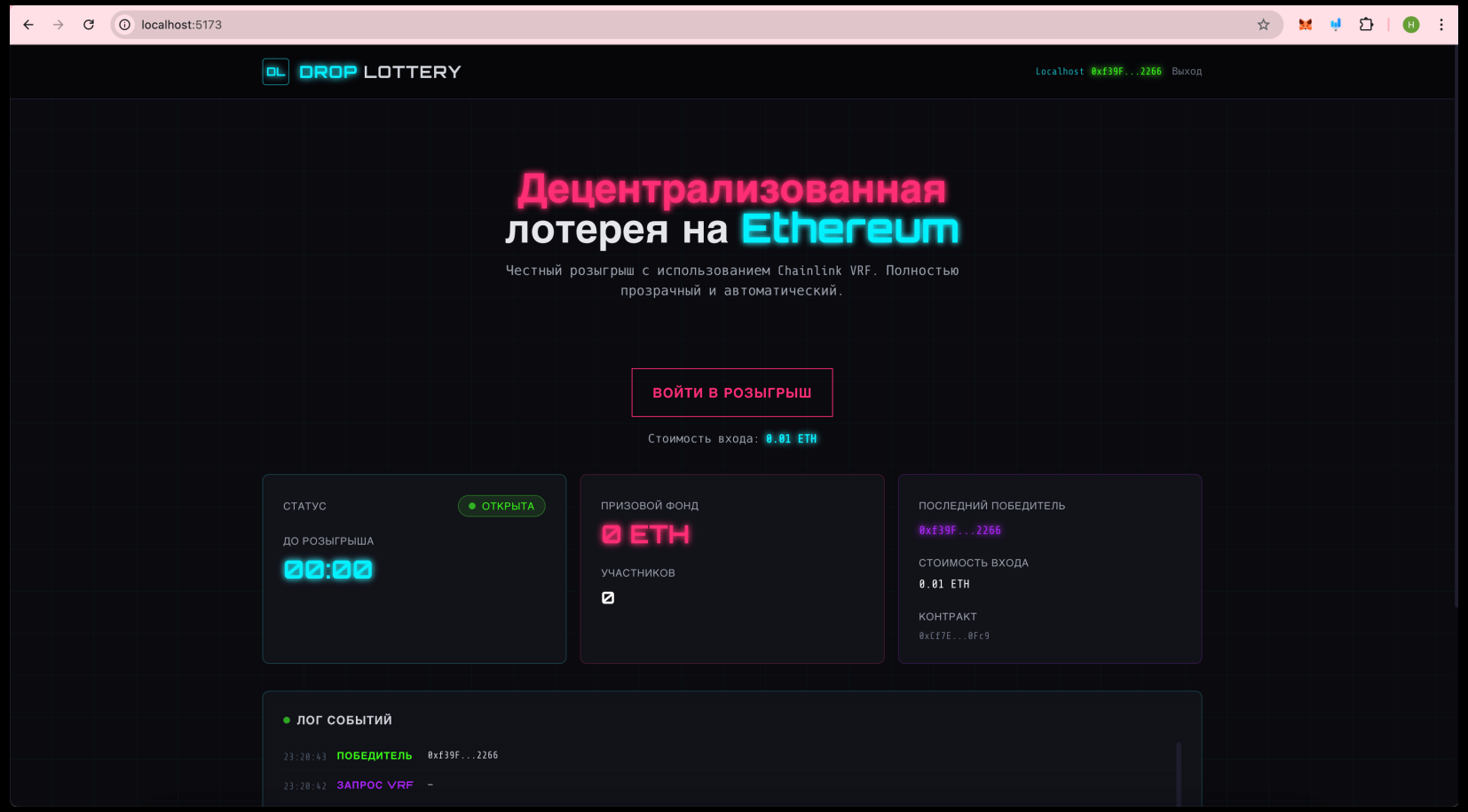
Обе библиотеки выбраны как простые и удобные решения без лишней сложности.

Скриншоты приложения



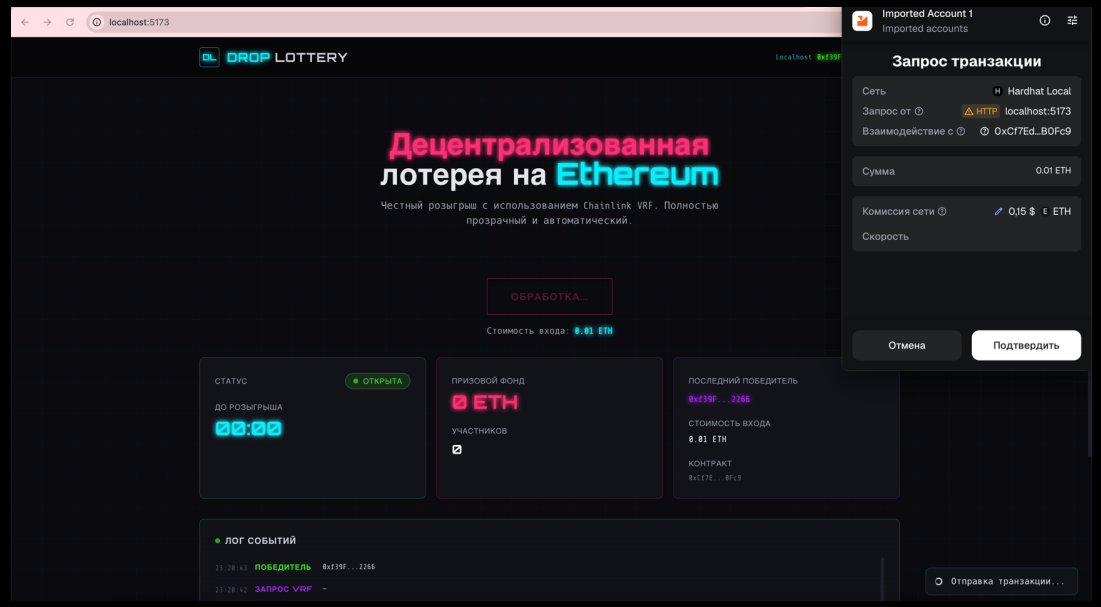
Стартовая страница

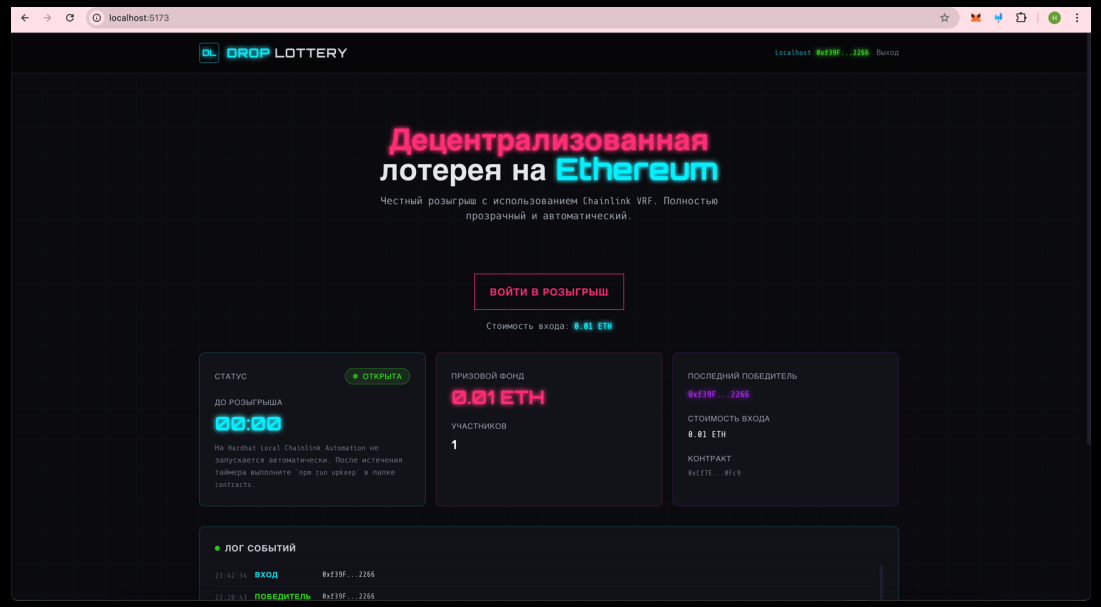
Скриншоты приложения



Состояние после подключения кошелька

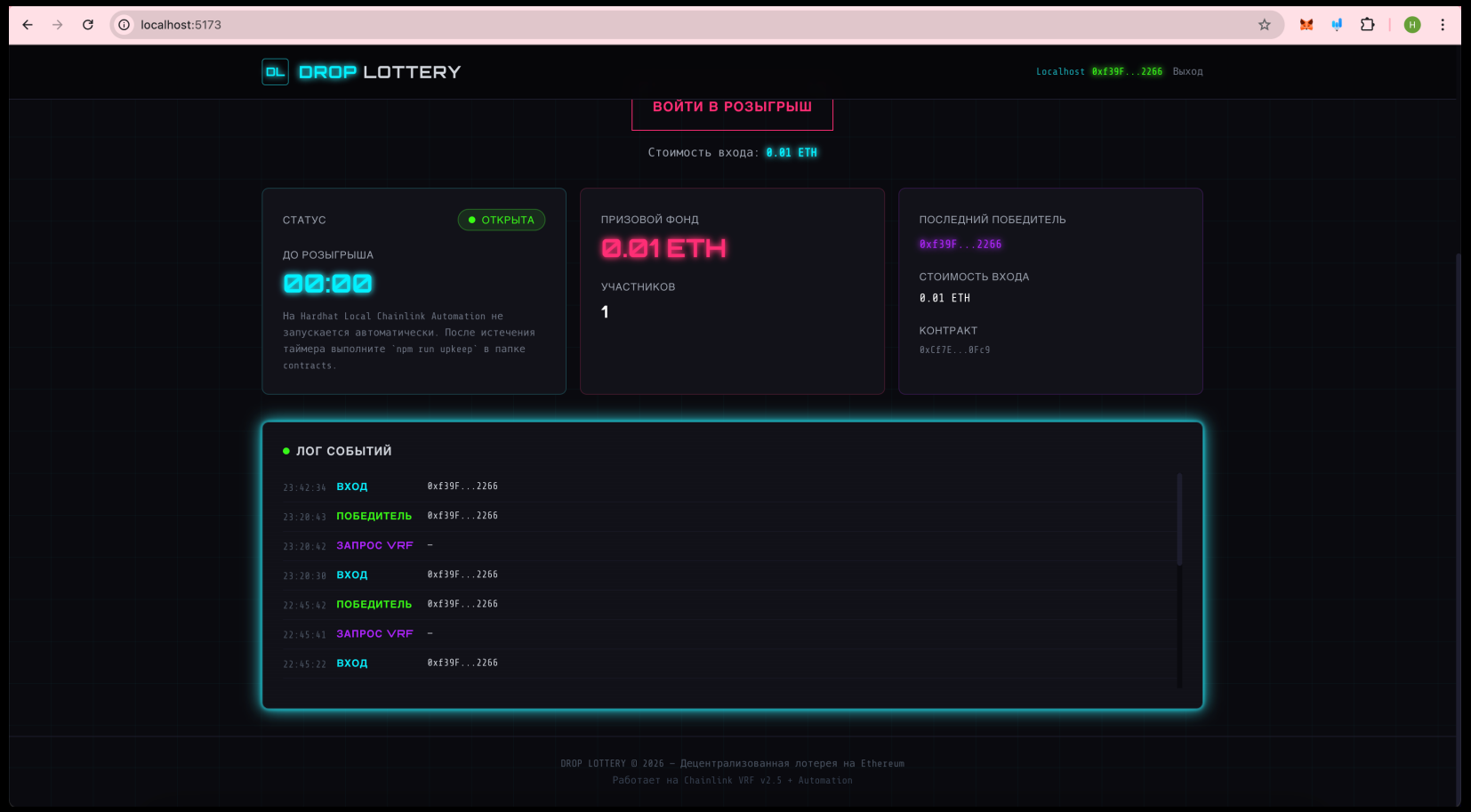
Скриншоты приложения





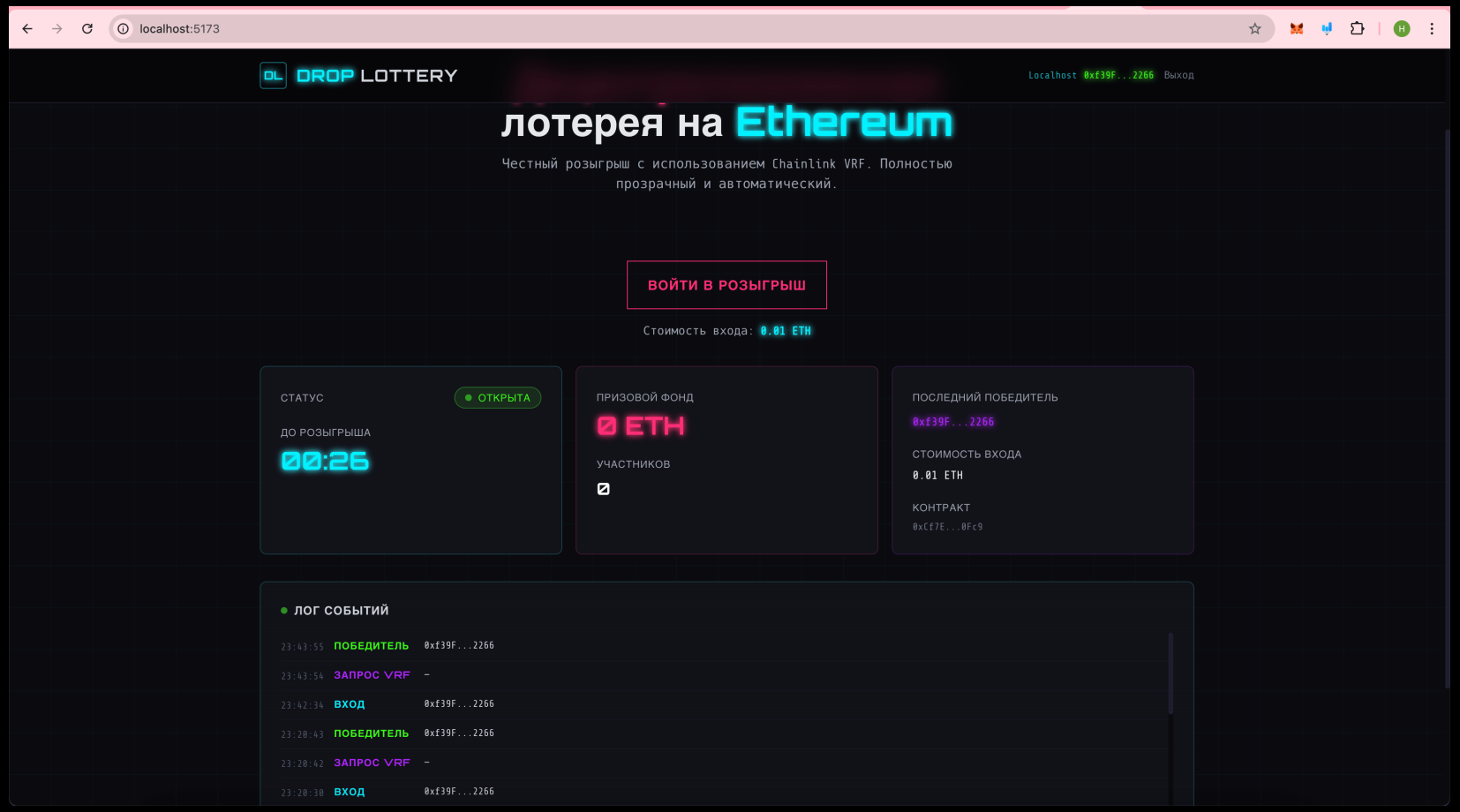
Участие в розыгрыше

Скриншоты приложения



Лог событий

Скриншоты приложения



После розыгрыша

Про интерфейс

Интерфейс отображает ключевые данные лотереи в реальном времени и позволяет пользователю взаимодействовать со смарт-контрактом через MetaMask.

Заключение

В ходе работы был реализован полноценный dApp с контрактом, локальной сетью, фронтендом и интеграцией с кошельком.

Проект показал, как связать Web3-часть и современный интерфейс в одном приложении.

Стек оказался довольно удобным: Hardhat упростил работу с контрактами, Ethers.js помог в взаимодействии с блокчейном, а React и TypeScript позволили собрать аккуратный и типобезопасный интерфейс.